MODULE 10 LAB · GRADING PREP

LibraryMind

Backend Deep-Dive & Lab-Review Survival Guide

Everything you need to understand, defend, and demo the LibraryMind backend — written to be understood, not memorised.

Prepared for **Caleb Gisa Mugisha**

AI-Powered Intelligent Library Assistant — Python Backend

Covers all 8 lab parts · full grading rubric · likely questions & answers · live-demo playbook

0 · How to use this guide

This guide walks through your actual backend code — file by file, concept by concept — and ties every piece to the grading rubric. It is built so that after one careful read you can **explain any part of the system in plain English**, not recite it. Read it in this order:

1. **The big picture** (§1) and **the request lifecycle** (§2) — this is the story you tell first in the review.
2. **The part-by-part walkthrough** (§3) — mapped one-to-one to the 8 lab parts and the rubric weights.
3. **The concepts cheat-sheet** (§4) — the "why" behind embeddings, RAG, token buckets, async, etc.
4. **Know your gaps** (§5) — honest weak spots and exactly how to talk about them. Reviewers reward honesty.
5. **Likely questions & answers** (§6) and the **live-demo playbook** (§7).
6. **Quick reference** (§8) — file map, env vars, and the numbers worth knowing cold.

> **The one-paragraph summary to open with.** "LibraryMind is an AI backend for a public library, built in four clean layers. At the top is a FastAPI REST API. Below it a service layer holds all the business logic — RAG, the chatbot, classification and summarisation. Below that, a provider layer wraps OpenAI, Anthropic and the AmaliAI gateway behind one interface with automatic failover. At the bottom, an infrastructure layer provides the ChromaDB vector store, a Redis cache, a token-bucket rate limiter and a usage/cost tracker. Each layer only knows about the one below it, so every component is independently testable — which is why the project has 286 tests at ~91% coverage."

Where your marks come from (memorise this table)

Rubric criterion	Weight	Where it lives in your code
RAG engine: grounded answers + sources	**20%**	`app/services/rag.py` , `app/prompts/rag.py`
Multi-provider AI layer with fallback	15%	`app/providers/` (base, resilient, retry, 3 providers)
Vector store seeded + semantic search	15%	`app/infrastructure/vector_store.py` , `scripts/seed_vector_store.py`
Multi-turn chatbot with memory	15%	`app/services/chatbot.py`
Classification + summarisation (valid JSON)	10%	`app/services/classifier.py` , `summariser.py` , `json_utils.py`
Caching, rate limiting, usage tracking	10%	`app/infrastructure/` (cache, rate_limiter, usage_tracker)
Clean, documented FastAPI endpoints	10%	`app/api/` (main, routers, middleware, schemas)
Code quality, structure, README	5%	whole repo: tests, mypy, ruff, docstrings

The RAG engine is the single biggest line item at 20%. Spend the most prep time there (§3, Part 4).

1 · The big picture: four layers

LibraryMind uses a **layered architecture**. The golden rule: **a layer may only call the layer directly beneath it**. Requests flow down; data flows back up. This separation is the reason every piece can be unit-tested in isolation with mocks.

Layer 1 — API (FastAPI). Receives HTTP, validates input with Pydantic, routes to a service, shapes JSON out. *Contains no business logic.* Files: `app/api/`.

Layer 2 — Services (business logic). RAG engine, chatbot, classifier, summariser, embedding service. Orchestrates retrieval + prompts + AI calls. Files: `app/services/`.

Layer 3 — AI Providers. One common interface over OpenAI, Anthropic and AmaliAI, plus the `ResilientAIService` that tries them in order with automatic failover. Files: `app/providers/`.

Layer 4 — Infrastructure. ChromaDB vector store, Redis cache, token-bucket rate limiter, usage/cost tracker, cache-key helpers. Files: `app/infrastructure/`.

Why this matters for grading

If a reviewer asks "why is your code structured this way?", the answer is: **separation of concerns and testability**. The router doesn't know how RAG works; the RAG service doesn't know which AI vendor answered; the vendor code doesn't know there's a cache. You can swap Redis for memcached, or add a fourth AI provider, by touching exactly one layer.

Cross-cutting glue you should be able to name:

- **Settings** (`app/core/settings.py`) — one Pydantic `Settings` object, loaded from `.env`, validates that at least one provider key exists at startup.
- **Exceptions** (`app/core/exceptions.py`) — a small tree (`LibraryMindError` → provider / rate-limit / validation errors) that the API layer maps to HTTP status codes.
- **Logging** (`app/core/logging.py`) — structured logging via `structlog`; every request gets a UUID request-ID.
- **Dependency injection** (`app/api/dependencies.py`) — `lru_cache` factories that build each component once and share it (singletons).

2 · The request lifecycle (tell this story)

The clearest way to prove you understand the system is to narrate what happens for one request. Use `POST /search/ask` — the RAG endpoint — because it touches every layer. A patron asks: *"Recommend a book about space exploration."*

#	What happens	Which layer / file
1	FastAPI validates the body against `AskRequest` (question 5–1000 chars). Bad input → automatic **422**.	API · `schemas/search.py`
2	Router calls `RAGService.answer(question)`.	API · `routers/search.py`
3	**Cache check first.** A deterministic key is built from the model + normalised question. Cache hit → return instantly, `cached=True`, no AI call, no rate budget spent.	Service · `rag.py` + `cache.py`
4	**Then** the rate limiter is consulted (`acquire()`). Empty bucket → **429**. (Cache hits skip this on purpose.)	Infra · `rate_limiter.py`
5	Embed the question into a vector (embedding cache checked first).	Service · `embedding.py`
6	Query ChromaDB for the top-K (4) most similar books; convert distance→similarity.	Infra · `vector_store.py`
7	Drop any result below the relevance threshold (0.35). **If none survive → return a fixed refusal message, no AI call.**	Service · `rag.py`
8	Format survivors into a numbered context block; build the prompt (system rules + context + question).	Prompts · `prompts/rag.py`
9	Send through `ResilientAIService.generate()` → tries AmaliAI, falls back to OpenAI, then Anthropic.	Providers · `resilient.py`
10	Record token usage + estimated cost.	Infra · `usage_tracker.py`
11	Cache the answer (TTL 1 hour) and return `{answer, sources[], cached:false}`.	Service → API

Two ordering decisions worth pointing out unprompted (they signal real understanding):

1. **Cache before rate limit.** A repeat question is a free read — charging the limiter for it would punish honest users.
2. **Refusal is deterministic and skips the AI entirely.** If retrieval finds nothing relevant, you return a fixed sentence without calling any model. That means *no provider misbehaviour can ever fabricate an answer* — the anti-hallucination acceptance test passes by construction, not by luck.

3 · Part-by-part walkthrough

Each part below states what the lab asked, what you actually built, the key files, the core idea in code, and the questions a reviewer is most likely to ask.

Part 0 — Environment & configuration setup

What you built. A single typed `Settings` class (`pydantic-settings`) that loads from `.env` , with a `get_settings()` singleton cached by `lru_cache` . A `model_validator` refuses to start the app unless at least one provider key is present.

```
@model_validator(mode="after")
def _require_at_least_one_provider_key(self) -> Settings:
    if not self.configured_providers:
        raise ValueError("No AI provider configured. Set at least one of "
                         "OPENAI_API_KEY, ANTHROPIC_API_KEY, or AMALIAI_API_KEY...")
    return self
```

Key idea to explain: `configured_providers` returns the providers that actually have keys, *primary first*. That single property drives both startup validation and the failover order. `.gitignore` excludes `.env` , `venv/` , `__pycache__/` .

Part 1 — Multi-provider AI layer 15%

What the lab asked. One common interface for all providers; concrete OpenAI + Claude (or AmaliAI) classes; retry with exponential backoff; a `ResilientAIService` that tries providers in order and falls back; `RuntimeError` if all fail.

What you built.

- **The interface** — `AIProvider` is a `typing.Protocol` (structural typing) in `base.py` with `generate()` , `generate_chat()` and `embed()` . Results come back as a frozen `GenerationResult` dataclass carrying text + token counts.
- **Three concrete providers** — `OpenAIProvider` (official async SDK), `AnthropicProvider` (official async SDK), `AmaliAIProvider` (raw `httpx` to the AmaliAI gateway).
- **Shared retry policy** — `retry.py` builds a `tenacity` decorator: `stop_after_attempt(3)` , `wait_exponential(min=2, max=30)` , and `before_sleep_log(WARNING)` so retries are visible in logs.
- **The orchestrator** — `ResilientAIService` holds an ordered list, tries each, catches `ProviderError` , logs a warning, moves on; raises `AllProvidersFailedError` if all fail.

> **The two cleverest design points here:**
>
> - **The orchestrator itself satisfies the `AIProvider` protocol.** It has `generate / generate_chat / embed` with the same signatures, so callers literally cannot tell whether they're talking to one provider or a failover chain. That's why services just depend on "an AI service" and never mention vendors.
> - **Transient vs permanent errors.** Each provider defines a `_TRANSIENT` tuple (rate limits, timeouts, connection drops, 5xx). Those get retried. A bad API key is *not* transient — it propagates immediately, gets converted to `ProviderUnavailableError` , and the orchestrator falls straight through to the next provider. This is exactly what makes the "invalidate the primary key → auto-fallback" acceptance test pass.

Why `AllProvidersFailedError` multiply-inherits from `RuntimeError`: the lab literally says "a `RuntimeError` is raised." Your class is `class AllProvidersFailedError(ProviderError, RuntimeError)` — so `isinstance(exc, RuntimeError)` is true (satisfies the brief) *and* it stays in your own exception tree so the API maps it to HTTP 503.

Your live setup: `PRIMARY_PROVIDER=amaliai`, all three keys present → failover order is **AmaliAI → OpenAI → Anthropic**. AmaliAI is a gateway that proxies to OpenAI using model `gpt-4o-mini` and authenticates with an `X-Api-Key` header plus a `Provider` header (not `Authorization: Bearer`).

Part 2 — Core infrastructure part of 10%

Cache (`cache.py` + `keys.py`)

An async Redis wrapper (`redis.asyncio`) with `get/set/delete/ping`. Values are serialised with `orjson`. **Graceful degradation** is the headline feature: if Redis is down (or `cache_enabled=False`), the client is `None` and every method becomes a safe no-op — the app keeps working, just without caching. Every Redis call is wrapped in `except RedisError` that logs a warning and returns `None`.

Deterministic keys (`keys.py`): every key is `{version}:{scope}:{sha256(json(parts))}`. Using `json.dumps(..., sort_keys=True)` before hashing gives a canonical, collision-resistant encoding — identical inputs always produce the same key. The `version` prefix (`v1`) lets you invalidate a whole scope when a prompt changes, without flushing all of Redis.

Rate limiter (`rate_limiter.py`)

A **token-bucket** limiter. The bucket starts full (`burst` tokens) and refills at `requests_per_minute / 60` tokens per second. Each `acquire()` removes one token; an empty bucket raises `RateLimitExceededError`. Refill is computed from elapsed wall-clock time (`time.monotonic()`) so no one ever sleeps — you either get a token now or are rejected now.

```
def _refill(self) -> None:
    now = time.monotonic()
    elapsed = now - self._last_refill
    self._tokens = min(self._burst, self._tokens + elapsed * self._rate)
    self._last_refill = now
```

Thread-safety: protected by an `asyncio.Lock` (not a thread lock) because FastAPI runs all handlers on one event loop — an async lock is the correct, cheaper choice. Defaults: 60 requests/min, burst 10.

Usage tracker (`usage_tracker.py`)

An in-memory list of immutable `UsageRecord`s. `record()` looks up `(provider, model)` in a `PRICING` table (USD per 1k input/output tokens), computes the cost, and appends. `daily_cost_usd()` and `total_requests_today()` feed the `/health` endpoint. Unknown models cost `$0.00` so an unpriced provider never crashes the tracker.

> **Know this for the demo:** AmaliAI is intentionally priced at `$0` (training credits). So if you demo with AmaliAI as primary, `/health` shows `daily_cost_usd: 0.0` even though calls were made (the request *count* still increases). To show a non-zero dollar figure, either set `PRIMARY_PROVIDER=openai` for the demo, or simply explain the $0 is a deliberate pricing-table choice. See §5.

Part 3 — Knowledge base, embeddings & vector store 15%

Data: `app/data/books.json` holds **22 books** across **6 genres** (Science Fiction 5, Classic 5, Fantasy 4, Mystery 3, Non-Fiction 3, Historical Fiction 2) — comfortably over the "20 books, 5 genres" requirement. Each book has `id, title, author, year, genre, description`.

Embedding service (`embedding.py`)

Wraps the resilient AI service to produce vectors, with a Redis cache. `embed_one()` caches single texts; `embed_many()` checks the cache per-text, batches all *misses* into one API call, caches each result, and returns vectors in the original order. The cache key includes the model name, so changing the embedding model auto-invalidates old vectors. TTL is 24 hours.

Vector store (`vector_store.py`)

A thin wrapper over a **ChromaDB persistent collection** created with `{"hnsw:space": "cosine"}`. Two operations: `upsert()` (insert-or-update) and `search()` (top-K nearest).

> **The single most-likely "gotcha" question — distance vs similarity.**
> ChromaDB returns cosine **distance** in [0, 2] where **lower = more similar**. Your public API uses **similarity** in [0, 1] where **higher = more relevant**. You convert exactly once, at the boundary of `search()`:
>
> ```
> similarity = max(0.0, 1.0 - distance)
> ```
>
> Because of this, the RAG threshold comparison `score >= 0.35` is a *similarity* comparison. No code above the vector store ever sees a raw distance. Be ready to say this sentence out loud — it's the classic exam trap from the lab's "Common Pitfalls" page.

Seed script (`scripts/seed_vector_store.py`)

Run with `python -m scripts.seed_vector_store`. It loads the JSON, builds an embedding text per book (`"{title} by {author}. {description}"`), embeds them all in one batched call, and upserts into ChromaDB. Re-running is safe — upserts are idempotent and embeddings are cached.

Part 4 — RAG engine 20% — the big one

RAG = Retrieval-Augmented Generation: instead of trusting the model's memory, you *retrieve* relevant documents from your own data and *augment* the prompt with them, so the model answers from facts you control. `RAGService.answer()` is the centrepiece. The pipeline (memorise this):

```
cache lookup ──hit──> return cached answer (no AI, no rate budget)
      | miss
      v
acquire rate-limit token
      v
embed question --> vector search (top_k=4) --> drop results below 0.35
      |
      |-- none left --> return REFUSAL_MESSAGE  (deterministic, no AI call)
      |
      v some left
format context block --> build prompt (system + context + question)
      v
ResilientAIService.generate(T=0.3, max_tokens=512)
      v
record usage --> cache answer (TTL 1h) --> return RAGAnswer
```

The prompt (`prompts/rag.py`) is the source of the 20%. Its system message gives the model exactly one job — "answer using ONLY the books in the context" — and one fallback — reply verbatim with the refusal sentence. It tells the model to cite books by exact title and to never invent titles, authors or facts.

The response is an immutable `RAGAnswer` with: `answer` text, a `sources` list (title, author, similarity score), a `cached` flag, and `avg_relevance` .

> **Why this design wins each acceptance criterion:**
>
> - *"mentions relevant books"* → context block is built from real retrieved books, and the prompt forces grounding.
> - *"sources list with scores"* → every survivor becomes a `SourceCitation` .
> - *"off-topic → polite refusal not fabrication"* → deterministic refusal, no AI call.
> - *"same question twice is faster"* → cache hit on the second call.
> - *"usage shows cost of first call but not the cached one"* → `record()` only runs on a miss; cache hits and refusals never record usage.

Shared retrieval: `retrieve_context()` is split out from `answer()` so the chatbot can reuse the exact same embedding + search + filter logic without duplicating it.

Part 5 — AI librarian chatbot 15%

What you built. `ChatbotService` plus an in-memory `ConversationStore` (a `dict` mapping `conversation_id → list[Message]`). Per turn it:

1. Pulls the **recent** history — `store.recent(cid, N)` where `N = chat_history_max_messages` (10). This truncation is how you stay inside the model's context window.
2. Calls `RAGService.retrieve_context()` to ground the reply in catalogue books (no duplicated search logic).
3. Builds the messages list: `[system + catalogue context] + [history] + [new user turn]` .
4. Calls `generate_chat()` , then appends both the user message and the assistant reply to the store.

> **The three acceptance points and how they're met:**
>
> - *Memory works* ("tell me more about that one") → prior turns are sent to the model in the messages list.
> - *Different IDs isolated* → each `conversation_id` is a separate list; histories never bleed across IDs.
> - *No fabrication on greetings* → the prompt lets the bot chat naturally for "Hi!" but forbids inventing book titles/authors when there's no catalogue match.

Why in-memory is fine here: the lab explicitly allows it ("a simple list is sufficient"). You document that a distributed deployment would swap the dict for Redis. `list.append` is atomic under CPython's GIL, so concurrent appends are safe in-process.

Part 6 — Classification & summarisation 10%

Ticket classifier (`classifier.py` + `prompts/classifier.py`)

Sends the ticket with a low **temperature 0.1** (deterministic), using **4 few-shot examples** that cover the commonly-confused categories. The response is parsed by `parse_ai_json()` then validated into a `TicketClassification` Pydantic model whose fields are `Literal` enums — so an out-of-range value raises a `ValidationError` (surfaced as `ProviderError` → 503) rather than being silently accepted.

Review summariser (`summariser.py` + `prompts/summariser.py`)

Accepts 1–50 reviews, formats them as a numbered block, and prompts the model to synthesise **holistically** (patterns across all reviews, not one-by-one). Returns `overall_sentiment, estimated_rating (1-5), themes[], praise[], criticism[], recommendation`. Temperature 0.3.

> **The JSON-fence pitfall (called out in the lab) — and your fix.** Models often wrap JSON in ```` ```json … ``` ```` fences. `json_utils.parse_ai_json()` strips those fences with a regex, then calls `json.loads` ; on failure it raises a `ProviderError` that carries the raw response and the cleaned string in its `detail` for debugging. Both the classifier and summariser route every response through it. This is "the single most common failure" the brief warns about — and you handled it deliberately.

Defensible design choice — "mixed" sentiment: the summariser allows `overall_sentiment="mixed"` in addition to positive/neutral/negative, because evenly-split reviews genuinely are mixed. You note in the docstring that the API layer could map `mixed → neutral` if a strict three-value enum were required.

Part 7 — REST API 10%

App factory (`main.py`): `create_app()` wires settings → logging → FastAPI instance → middleware → exception handlers → routers. Using a factory (not a module-level app with side effects) keeps construction testable.

Method & path	Router	Request schema
POST `/search/books`	search	`SearchBooksRequest` (query 3–500, limit 1–50)
POST `/search/ask`	search	`AskRequest` (question 5–1000)
POST `/chat`	chat	`ChatRequest` (conversation_id 1–64, message 1–4000)
POST `/classify/ticket`	classify	`ClassifyTicketRequest` (text 5–4000)
POST `/summarise/reviews`	summarise	`SummariseReviewsRequest` (1–50 reviews)
GET `/health`	health	— (returns status, daily cost, request count)

Validation: every request body is a Pydantic model with `extra="forbid"` and min/max lengths, so empty or malformed input returns **422** automatically. **Error mapping** (`middleware.py`): handlers map `RateLimitExceededError→429` , `AllProvidersFailedError→503` , `ProviderError→503` , and a generic `LibraryMindError→500` . Handlers are registered *narrowest-first* so FastAPI matches the most specific type.

Other production touches: CORS middleware enabled; a `RequestIDMiddleware` stamps every request with a UUID (bound into structured logs and returned as `X-Request-ID`); `ORJSONResponse` for fast serialisation; Swagger UI at `/docs` .

Part 8 — Testing & quality 5%

286 test functions, ~91% line coverage. Tests mirror the package structure (`tests/providers` , `tests/services` , `tests/infrastructure` , `tests/api` , `tests/prompts`). There's also a `scripts/smoke_test.py` for end-to-end manual validation. The repo uses `ruff` (lint/format), `mypy` (types), and `pre-commit` hooks. This is what earns the code-quality marks.

4 · Concepts cheat-sheet (understand, don't cram)

Embeddings & semantic search

An **embedding** turns text into a list of numbers (a vector) that captures its meaning. Texts with similar meaning land close together in vector space. "Semantic search" = embed the query, then find the nearest stored vectors. That's why searching "space travel adventure" finds sci-fi books even if those exact words never appear — meaning, not keywords.

Cosine distance vs similarity

Cosine measures the angle between two vectors. **Similarity** = 1 means same direction (same meaning); 0 means unrelated. **Distance** = 1 − similarity, so 0 means identical. ChromaDB gives you distance; you convert to similarity with `max(0, 1 - distance)`. Mixing these up (filtering "distance ≥ 0.35" instead of "similarity ≥ 0.35") would discard your best matches — which is exactly the trap you avoided.

RAG in one breath

Retrieve relevant facts from your own store → stuff them into the prompt → tell the model to answer only from them → it answers grounded, with citations, and admits when it doesn't know. It fixes the two big LLM problems: stale knowledge and hallucination.

Token-bucket rate limiting

Picture a bucket of tokens that refills steadily. Each request spends one token. Empty bucket → reject. The `burst` size lets you absorb short spikes; the refill rate caps the sustained average. It protects both your budget and the upstream provider's rate limits.

Retries with exponential backoff

Transient failures (a rate-limit blip, a timeout) often succeed if you wait and retry. "Exponential backoff" doubles the wait each attempt (2s, 4s, 8s…) so you don't hammer a struggling service. You retry 3 times; if still failing, you give up and let the orchestrator fall back to the next provider. Permanent errors (bad key) aren't retried.

Protocol vs abstract base class

A `Protocol` defines a shape ("anything with these methods counts"). Unlike an ABC, a class doesn't have to inherit from it — it just has to *match*. This is "structural typing / duck typing with type-checker support." It's why your `ResilientAIService` can stand in for an `AIProvider` without inheriting anything, and why providers are trivial to mock in tests.

Dependency injection & the `lru_cache` singletons

Instead of building Redis/Chroma/services inside the code that uses them, you build them in `dependencies.py` and FastAPI passes them in via `Depends()`. Each factory is `@lru_cache(maxsize=1)` so the heavy objects (DB clients, conversation store) are created once and shared. Tests override these with mocks via `app.dependency_overrides`.

Why async/await everywhere

AI and network calls spend most of their time *waiting*. `async` lets the server handle other requests during that wait instead of blocking a thread. That's why providers, services and the cache are all `async`.

Pydantic validation

Pydantic models validate and coerce data at the boundary. Request schemas reject bad input before it reaches your logic (→ 422). Response models (like `TicketClassification`) guarantee the output shape and, via `Literal` enums, catch a model that returns an unexpected category.

Graceful degradation

A non-essential dependency failing should degrade, not crash. If Redis is down, caching silently turns off and the app still answers. The cache is "best-effort, never a correctness guarantee."

5 · Know your gaps (honesty scores points)

Reviewers respect a candidate who knows the limits of their own system. Here are the honest weak spots and exactly how to talk about each. If asked, lead with the fact, then the rationale, then the fix.

Gap 1 — Rate limiting & usage tracking only run on the RAG path.
In the code, `rate_limiter.acquire()` and `usage_tracker.record()` are called *only inside* `RAGService.answer()` (i.e. `POST /search/ask`). The `/chat`, `/classify/ticket`, `/summarise/reviews` and `/search/books` endpoints don't currently invoke them, even though some service docstrings describe an intended "applied at the API layer" wiring.

How to answer: "The production patterns are fully implemented and demonstrated end-to-end on the RAG path, which is the primary, most expensive AI route. The limiter and tracker are real, injected, and tested. Extending them to the other routers is a small, uniform change — either a shared FastAPI dependency that acquires a token and records usage, or moving both into the resilient service. I scoped them to the RAG flow first because that's where the acceptance criteria (429 on overload, cost on first call but not the cached one) are demonstrated." That's accurate and shows you understand the architecture.

Gap 2 — With AmaliAI as primary, `/health` reports `$0.00` cost.
AmaliAI isn't in the `PRICING` table (training credits → $0), so cost sums to zero even though the request count rises.

How to answer / demo: "Cost is computed from a per-model pricing table; AmaliAI is deliberately priced at $0 because it runs on training credits. The tracking mechanism works — request counts increment and OpenAI/Anthropic calls would show real cents. To demonstrate a non-zero figure I can set `PRIMARY_PROVIDER=openai`." For the live review, consider switching primary to OpenAI so the cost line is visibly non-zero (the rubric note says "non-zero cost after at least one AI call").

Gap 3 — Minor: embedding-model naming. The seed script labels embeddings with `amaliai_embedding_model` while the runtime `EmbeddingService` uses `openai_embedding_model`. Both default to `text-embedding-3-small`, so the vectors are the same model and search works — but the cache-key label differs. If asked, say: "Both point to `text-embedding-3-small`, so query and stored vectors share the same embedding space; I'd unify the setting name to remove the cosmetic mismatch." (The lab's "embedding model mismatch" pitfall is about *changing* models — which you'd handle by re-seeding.)

Gap 4 — Conversation memory is in-process. Not really a gap — the lab allows it — but be ready: "Histories live in a dict, so they reset on restart and don't span multiple workers. That's the documented lab-scope simplification; production would back it with Redis."

6 · Likely questions & model answers

Short, confident, defensible. Answer in your own words — these are the shape of a good answer, not a script.

Architecture & design

Q. Walk me through your architecture.

A. Four layers, each only calling the one below: FastAPI (routing + validation), services (business logic — RAG, chat, classify, summarise), AI providers (one interface with failover), and infrastructure (vector store, cache, rate limiter, usage tracker). Separation makes each piece independently testable — hence 286 tests at ~91% coverage.

Q. Why a layered design instead of putting logic in the routers?

A. Routers should only validate and shape JSON. Keeping business logic in services means I can test logic without HTTP, swap infrastructure without touching services, and reason about one concern at a time.

Q. How do you inject dependencies, and why singletons?

A. `dependencies.py` has `@lru_cache` factories that FastAPI passes in via `Depends()`. Singletons because the DB clients and conversation store are stateful and expensive — build once, share. Tests override them with `app.dependency_overrides`.

Multi-provider layer

Q. How does failover actually work?

A. `ResilientAIService` holds providers in order (primary first). It tries each: on a `ProviderError` it logs a warning and moves to the next. If all fail it raises `AllProvidersFailedError`. The orchestrator itself implements the provider interface, so callers can't tell it's a chain.

Q. Show me where retry/backoff happens, and what gets retried.

A. In `retry.py`, a tenacity decorator: 3 attempts, exponential wait 2→30s, with a WARNING logged before each sleep so retries are observable. Only transient errors (rate limits, timeouts, connection drops, 5xx) are retried. A bad API key isn't transient — it propagates and triggers fallback.

Q. If I invalidate the primary key, what happens?

A. The primary raises an auth error, converted to `ProviderUnavailableError`; the orchestrator catches it, logs the failure, and the next provider answers — no crash. That's the Part 1 acceptance test.

Q. Why is AmaliAI driven with httpx instead of an SDK?

A. It's an OpenAI-compatible gateway with its own auth (`X-Api-Key` + a `Provider` header), not the OpenAI auth scheme. A thin httpx client handles both chat and embeddings without pulling in a vendor SDK that wouldn't match the gateway's headers.

Vector store & RAG

Q. Distance or similarity — which does ChromaDB return, and what do you do?

A. ChromaDB returns cosine *distance* in [0,2], lower = more similar. I convert to similarity with `max(0, 1−distance)` once, inside `search()`. Everything above the vector store works in similarity [0,1], so my threshold `≥ 0.35` is a similarity threshold.

Q. What stops the RAG engine from hallucinating on an off-topic question?

A. Two things. First, if no retrieved book clears the relevance threshold, I return a fixed refusal sentence without calling the model at all. Second, even when books are found, the system prompt forbids using anything outside the provided context and tells it to refuse if the context is insufficient.

Q. How do you prove a cache hit costs nothing?

A. `record()` is only called after a successful generation on a cache miss. Cache hits return before that line and never touch the rate limiter either. So the second identical question is faster, spends no budget, and adds no usage record.

Q. What's in a source citation and why so minimal?

A. Title, author, similarity score — just what a patron needs to find the book. I deliberately leave out id and description; those belong on search hits, not on grounded citations.

Chatbot

Q. How does memory work and how do you avoid context overflow?

A. Each `conversation_id` maps to its own message list. Per turn I send only the most recent N (10) messages plus the new one, so the prompt stays within the context window no matter how long the chat grows.

Q. How are two conversations kept separate?

A. Different IDs are different lists in the store — history never crosses between them. There's a test asserting exactly that.

Q. Does the chatbot reuse RAG?

A. Yes — it calls `RAGService.retrieve_context()` for grounding, so embedding and search logic isn't duplicated. It just doesn't run the full `answer()` (no deterministic refusal), because a chatbot should still reply naturally to a greeting.

Classification, summarisation & JSON

Q. How do you guarantee valid JSON from the model?

A. Defence in depth: the prompt says "return ONLY JSON"; `parse_ai_json()` strips any markdown fences before `json.loads`; and a Pydantic model with `Literal` enums validates the fields. If anything is off, I raise a `ProviderError` that keeps the raw response for debugging.

Q. Why temperature 0.1 for classification but 0.3 for summarisation?

A. Classification is a closed-enum, deterministic task — low temperature maximises consistency. Summarisation needs slightly more fluency to write a readable recommendation, so 0.3, still grounded in the reviews.

Q. Why few-shot examples in the classifier?

A. A few labelled examples dramatically improve enum compliance and disambiguate confusable categories like technical vs complaint. The model sees the exact input→output shape before classifying.

Infrastructure & API

Q. What happens if Redis is down?

A. The app keeps working. The cache client is `None` or its calls are wrapped in a `RedisError` catch, so every cache op becomes a logged no-op. Caching is an optimisation, never a correctness requirement.

Q. How are cache keys built and why hash?

A. `{version}:{scope}:{sha256(canonical-json of parts)}`. Canonical JSON (sorted keys) makes identical inputs collide deterministically and different inputs not collide. The version prefix lets me invalidate one scope when a prompt changes without flushing Redis.

Q. Why an asyncio.Lock in the rate limiter, not a thread lock?

A. FastAPI runs handlers on a single event loop, so coroutines, not threads, are the concurrency unit. An async lock is correct and cheaper here. I documented that a multi-process deployment would need a Redis-backed limiter instead.

Q. How does an internal exception become the right HTTP code?

A. Registered exception handlers map my domain exceptions: rate limit → 429, provider/all-providers-failed → 503, generic → 500. Pydantic handles 422 for bad input automatically. Handlers are registered narrowest-first so the most specific one wins.

Q. What does `/health` tell you?

A. Status, version, which providers are configured, whether Redis is reachable (live ping), today's spend in USD, and today's request count. It never makes a paid AI call.

Curveballs

Q. Is every endpoint rate-limited?

A. Honestly, no — the limiter and usage tracker currently run on the RAG path (`/search/ask`), which is the main AI-spend route and where the acceptance criteria are shown. Extending them to the other routers is a small, uniform change via a shared dependency; it's a known, documented limitation rather than a missing capability. (See §5.)

Q. Your daily cost shows $0 — is tracking broken?

A. No. AmaliAI runs on training credits, priced at $0 in the table, so cost sums to zero while the request count still rises. With OpenAI as primary it shows real cents — I can switch and demo that.

Q. What was your hardest bug?

A. (Pick a real one. A strong, true-to-the-code example:) The distance-vs-similarity conversion — early on, filtering on raw ChromaDB distance with a "≥ threshold" test threw away the best matches. Once I converted to similarity at the vector-store boundary and treated 0.35 as a similarity floor, retrieval behaved. I also logged raw model output before parsing, which is how I caught the markdown-fence JSON issue.

7 · Live-demo playbook

If you're asked to run it, this is the order that demonstrates every acceptance criterion smoothly.

Setup

```
# 1. Activate venv and confirm config loads (validates a provider key exists)
python -c "from app.core.settings import get_settings;
print(get_settings().configured_providers)"

# 2. Seed the vector store (populates ChromaDB with the 22 books)
python -m scripts.seed_vector_store

# 3. Run the API with auto-reload, then open Swagger
uvicorn app.main:app --reload
#    -> browse http://localhost:8000/docs
```

Demonstrate each rubric item

To show…	Do this
Semantic search (15%)	`POST /search/books` with `"desert planet adventure"` → sci-fi books rank high, each with a score.
RAG grounded answer (20%)	`POST /search/ask` "Recommend a classic romance novel" → answer cites catalogue books; `sources` populated.
Anti-hallucination (20%)	`POST /search/ask` "What is the meaning of life?" → polite refusal, empty sources, no fabrication.
Cache hit	Send the same `/search/ask` twice → 2nd is instant and `cached:true`.
Chatbot memory (15%)	`/chat` turn 1 "Recommend a thriller"; turn 2 (same `conversation_id`) "Tell me more about that" → elaborates on turn 1's book.
Conversation isolation	Repeat with a different `conversation_id` → fresh context.
Classification (10%)	`/classify/ticket` "My library card isn't working at the self-checkout and I'm very frustrated" → `technical / high / negative`.
Summarisation (10%)	`/summarise/reviews` with 3–5 mixed reviews → balanced praise + criticism, valid JSON.
Rate limit → 429	Lower `RATE_LIMIT_PER_MINUTE` / `BURST`, hammer `/search/ask` → HTTP 429. (Limiter lives on the RAG path.)
Provider fallback (15%)	Temporarily break the primary key in `.env`, restart, call `/search/ask` → logs show the fallback; answer still returns.
Validation → 422	`/chat` with an empty `message` → 422 with field details.
Health / cost	`GET /health` → daily cost + request count. (For non-zero cost, set `PRIMARY_PROVIDER=openai`.)
Tests / quality (5%)	`pytest` → 286 passing; mention ~91% coverage, ruff, mypy.

Pre-flight checklist before the review: re-run the seed script (in case the Chroma store is stale), start the server once to confirm it boots, run `pytest` once, and decide your primary provider (OpenAI if you want a visible non-zero cost figure). Have `/docs` already open.

8 · Quick reference

Numbers worth knowing cold

Thing	Value	Where
Books / genres	22 books, 6 genres	`app/data/books.json`
Tests / coverage	286 tests, ~91%	`tests/`, `coverage.xml`
RAG top-K	4	`rag_top_k`
Relevance threshold (similarity)	0.35	`rag_relevance_threshold`
RAG temperature / max tokens	0.3 / 512	`prompts/rag.py`
Classifier temp / max tokens	0.1 / 300	`prompts/classifier.py`
Summariser temp / max tokens	0.3 / 600	`prompts/summariser.py`
Chatbot temp / history N	0.7 / last 10 msgs	`prompts/chatbot.py`, settings
Rate limit defaults	60/min, burst 10	`rate_limit_*`
RAG cache TTL / embedding TTL	1 hour / 24 hours	`rag.py`, `embedding.py`
Retry policy	3 attempts, backoff 2→30s	`providers/retry.py`
Primary provider (live)	amaliai → openai → anthropic	`.env`

Environment variables (the ones to recognise)

`PRIMARY_PROVIDER`, `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `AMALIAI_API_KEY`, `AMALIAI_BASE_URL`, `AMALIAI_CHAT_MODEL`, `AMALIAI_PROVIDER`, `CHROMA_PERSIST_DIR`, `CHROMA_COLLECTION_NAME`, `RAG_TOP_K`, `RAG_RELEVANCE_THRESHOLD`, `REDIS_URL`, `CACHE_ENABLED`, `RATE_LIMIT_PER_MINUTE`, `RATE_LIMIT_BURST`, `CHAT_HISTORY_MAX_MESSAGES`, `BUDGET_DAILY_LIMIT_USD`, `CORS_ALLOWED_ORIGINS`.

File map (point to these confidently)

Concern	File
App factory, middleware wiring	`app/main.py`
Settings & validation	`app/core/settings.py`
Exception tree	`app/core/exceptions.py`
Provider interface + result type	`app/providers/base.py`
Failover orchestrator	`app/providers/resilient.py`
Retry policy	`app/providers/retry.py`
Concrete providers	`openai_provider.py`, `anthropic_provider.py`, `amaliai_provider.py`
Cache + keys	`app/infrastructure/cache.py`, `keys.py`
Rate limiter	`app/infrastructure/rate_limiter.py`
Usage tracker + pricing	`app/infrastructure/usage_tracker.py`

Vector store	`app/infrastructure/vector_store.py`
RAG engine	`app/services/rag.py`
Chatbot + store	`app/services/chatbot.py`
Classifier / summariser	`app/services/classifier.py` , `summariser.py`
JSON fence parser	`app/services/json_utils.py`
Embedding service	`app/services/embedding.py`
Prompts (single source of truth)	`app/prompts/*.py`
Routers	`app/api/routers/*.py`
Request/response schemas	`app/schemas/*.py`
DI factories	`app/api/dependencies.py`
Seed script	`scripts/seed_vector_store.py`

> **Final confidence reminder.** Your code is genuinely strong: clean layering, real failover, deterministic refusal, defensive JSON parsing, graceful cache degradation, and a large, passing test suite. Lead with the architecture story, narrate the RAG lifecycle, be precise about distance-vs-similarity, and be honest about the one rate-limit/usage-tracking scoping gap. That combination — working system + clear understanding + honesty — is exactly what the rubric rewards. Good luck.